

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-206Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №1

Задача: Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант сортировки: Поразрядна сортировка.

Вариант ключа: даты в формате DD.MM.YYYY, например 1.1.1, 1.9.2009, 01.09.2009, 31.12.2009..

Вариант значения: числа от 0 до $2^{64} - 1$.

1 Описание

Требуется написать поразрядную сортировку. Основная ее идея заключается в сортировке чисел устойчивой сортировкой по каждому разряду, начиная с младшего или со старшего.

Воспользуемся следующей леммой из [1]: «Пусть имеется n d -значных чисел, в которых каждая цифра принимает одно из k возможных значений. Тогда алгоритм RADIX SORT позволяет выполнить корректную сортировку этих чисел за время $\Theta(d(n+k))$, если устойчивая сортировка, используемая данным алгоритмом, имеет время работы $\Theta(n+k)$ ».

2 Исходный код

На каждой непустой строке входного файла располагается пара «ключ-значение», поэтому создадим двумерный массив *arr*, в которой будем хранить ключ и значение. Первые три элемента во вложенном массиве будут хранить дату, на 4-м месте будет значение, а 5-й и 6-й - флаги, хранящие информацию о формате введенной даты. Программа начинается с цикла. В нем я читаю введенную строку и обрабатываю ее: сохраняю дату, месяц, год, значение, а так же проверяю формат даты. Затем я реализовал сортировку подсчетом, следуя объяснениям из [1]. В финальной части программы я вызываю сортировку подсчетом, в которой за ключем является число месяца, далее вызываю сортировку, где ключ - месяц, а затем - год. В конце вывожу измененный массив.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "radixsort.h"
5
6  int main(){
7      long long n = 1, size = 0, data1, data2, data3;
8      n = 1000000;
9      unsigned long long val;
10     int max_d = 31, max_m = 12, max_y = 9999;
11     char point;
12     char input[100];
13
14     unsigned long long (*arr)[6] = malloc(n * sizeof(unsigned long long[6]));
15     long long i = 0;
16
17     while(fgets(input, sizeof(input), stdin) != NULL){
18         if (i == n){
19             arr = realloc(arr, 2 * n * sizeof(unsigned long long[6]));
20             n *= 2;
21         }
22
23         if (input[0] == '\n'){
24             continue;
25         }
26
27         arr[i][4] = 0;
28         arr[i][5] = 0;
29
30         if (input[0] == '0'){
31             arr[i][4] = 1;
32             if (input[3] == '0'){
33                 arr[i][5] = 1;
34             }
35         }
```

```

36     else if (input[2] == '.' && input[3] == '0'){
37         arr[i][5] = 1;
38     }
39     else if (input[1] == '.' && input[2] == '0'){
40         arr[i][5] = 1;
41     }
42     sscanf(input, "%lld.%lld.%lld\t%llu", &data1, &data2, &data3, &val);
43     arr[i][0] = data1;
44     arr[i][1] = data2;
45     arr[i][2] = data3;
46     arr[i][3] = val;
47     i+=1;
48     size += 1;
49     memset(input, 0, sizeof(input));
50 }
51
52 radix_sort(arr, max_d, max_m, max_y, size);
53
54 for (long long i = 0; i < size; i++){
55     if (arr[i][4] == 1)
56         printf("0");
57     printf("%lld.", arr[i][0]);
58     if (arr[i][5] == 1)
59         printf("0");
60     printf("%lld.", arr[i][1]);
61     printf("%lld\t%llu\n", arr[i][2], arr[i][3]);
62 }
63 free(arr);
64 }

```

radixsort.c	
void counting_sort(unsigned long long (*arr)[6], long long max_val, long long size, long long key)	Функция сортировки подсчётом
void radix_sort(unsigned long long (*arr)[6], long long max_d, long long max_m, long long max_y, long long size);	Функция поразрядной сортировки

3 Консоль

```
dmitrykochev$ cat test1.txt
1.1.1    13207862122685464576
1.1.1    4588010303972900864
01.02.2008      7670388314707853312
01.02.2008      12992997081104908288
```

```
1.2.2    998010303972900861
1.1.2    998010303972900869
1.1.3    778010303972900862
2.2.2    888010303972900861
1.2.2    998010303972900863
2.2.12   998010303972900869
```

```
02.2.2    888010303972900860
1.02.2    998010303972900862
```

```
dmitrykochev$ ./labc <test1.txt
1.1.1    13207862122685464576
1.1.1    4588010303972900864
1.1.2    998010303972900869
1.2.2    998010303972900861
1.2.2    998010303972900863
1.02.2    998010303972900862
2.2.2    888010303972900861
02.2.2    888010303972900860
24.2.2    998010303972900865
1.1.3    778010303972900862
2.2.12   998010303972900869
01.02.2008      7670388314707853312
01.02.2008      12992997081104908288
```

4 Тест производительности

Тесты производительности представляют из себя следующее: поразрядная сортировка, которая внутри себя использует сортировку подсчетом, сравнивается с сортировкой qsort. Для второй я буду переводить дату в число по некоторому правилу, так я смогу отсортировать их стандартной сортировкой. Время чтения элементов и создания массивов не учитывается. Тестов будет три: на 2^{22} , 2^{20} , 2^{18} и 2^{16} элементов.

```
dmitrykochev$ ./benchmark2 <test_data_0.txt
Count of lines is 4194304
Radix sort time: 0.7720
Qsort time: 0.8570
dmitrykochev$ ./benchmark2 <test_data_1.txt
Count of lines is 1048576
Radix sort time: 0.3000
Qsort time: 0.2040
dmitrykochev$ ./benchmark2 <test_data_2.txt
Count of lines is 262144
Radix sort time: 0.0540
Qsort time: 0.0450
dmitrykochev$ ./benchmark2 <test_data_3.txt
Count of lines is 65536
Radix sort time: 0.0170
Qsort time: 0.0120
```

Как можно заметить, при увеличении числа элементов поразрядная сортировка становится эффективнее qsort.

Qsort работает за $\Theta(n \log(n))$ (где n - количество элементов). Поразрядная сортировка работает, как было оговорено ранее, за $\Theta(d(n+k))$. При уменьшении количества элементов qsort вырывается вперед.

Это связано с тем, что линейная сложность при большом количестве данных становится очень эффективной. Можно сказать, что существует число n_0 такое, что все тесты, в которых количество элементов больше, чем n_0 , будут сортироваться быстрее поразрядной сортировкой, чем qsort для нескольких ключей. Наоборот же, чем меньше количество данных, тем лучше себя проявляет qsort в сравнении с поразрядной сортировкой, это видно из асимптотики алгоритмов.

5 Выводы

Выполнив первую лабораторную работу по курсу «Дискретный анализ», я научился писать и тестировать поразрядную сортировку. Безусловно, это дало мне ценный опыт в написании алгоритма своими руками с нуля. Я столкнулся с нетипичными проблемами, которые пришлось решать в одиночку. Это очень важно. Также я впервые написал свой бенчмарк, что также очень полезно. Это лабораторная работа показала, как должна выглядеть любая работа программиста: наличие тестов, проверка кода на работоспособность, сравнение с аналогами. Я буду стараться делать так в будущем в любых своих проектах.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* -- Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. -- 1296 с. (ISBN 5-8459-0857-4 (рус.))